

Glenns-webshop

- Vi utvecklade vår webshop från tidigare arbete
- Vi tog inspiration från webhallen
- Vi är: Jonas, Pontus, Adam, Syed, Fredrik, grupp 5
- Vi hade ungefär 4 veckor på oss
- Två kravlistor, en grundläggande och en avancerad
- Den första kravlistan uppfylldes redan efter 1 sprint
- Resten av tiden bockade vi av den avancerade listan
- Vi hade kort och sprint, men egentligen handlade det mest om att någon tog en punkt på den publicerade listan
- Konflikter? En icke avslutat konflikt mellan Stockholm och Göteborg

Glenns-webshop

- User stories? Vi skapade user stories på vårt uppstartsmöte
- Sprint backlog: ändrade user stories till aktiviteter
- Mötestruktur: Vi hade 1 timmes möten varannan dag
- Konflikter? pågående konflikt mellan Stockholm och Göteborg
- Sammanfattning: bygg modulärt, avgränsa, ta ett problem i taget
- Lärdomar: fråga om du inte fattar
- Lärdomar: förklara tydligt, fel uppenbarar sig när man förklarar

Glenns-webshop

- Webshopen består av tre delar
- `app/page.tsx`, `components/*` (`lib`, `server`, `types`)/`*`
- `Page.tsx` är huvudsidan som i sin tur importerar `components` etc
- `Components` innehåller all funktionalitet, ex paginering, sök, login etc
- `Lib`: funktioner för databas, autentisering etc
- `Server`: produkt data (json), senare på remote supabase
- `Types`: exportering av typerna i databasen

Glenns-webshop

- Databas server: supabase
- Server: vercel
- Struktur av projektet: välj vilken del du vill lösa
- Vad fungerade bra? samarbetet
- Stolt över? Snabb utveckling av sidan
- Svårighet? Hitta en gratis epostleverantör
- Lärdomar? Avgränsa så mycket som möjligt
- Mer tid? Användar recensioner, erbjudanden, betallösning, utveckla designen

Glenns-webshop

- Komponenterna:

Search: Använder sig av useState, en hook som initializerar en state variabel:

```
const [query, setQuery] = useState(""); query -> "" setQuery -> q
```

```
onChange={(e) => { const value = e.target.value; setQuery(value); searchProducts(value);
```

setQuery updateras med value

Notera att implementera ett state management inte är trivialt, useState ger detta gratis

Anv. skriver → state updatering → q initieras → produkt filtrering → res till nästa instans

Glenns-webshop

- Komponenterna:

pagination: Använder sig av useTransition, en hook som följer förändringar i websidan i ux:

```
const [isPending, startTransition] = useTransition();
```

IsPending → boolean, om sann, startTransition → "is loading ..."

```
StartTransition(() => { router.push(`${pathname}?${params.toString()}`, { scroll: false, });  
});
```

Ex: click → startTransition → ispending true → "loading ..." → låser buttons etc

UseTransition: "snart är du där, håll ut"

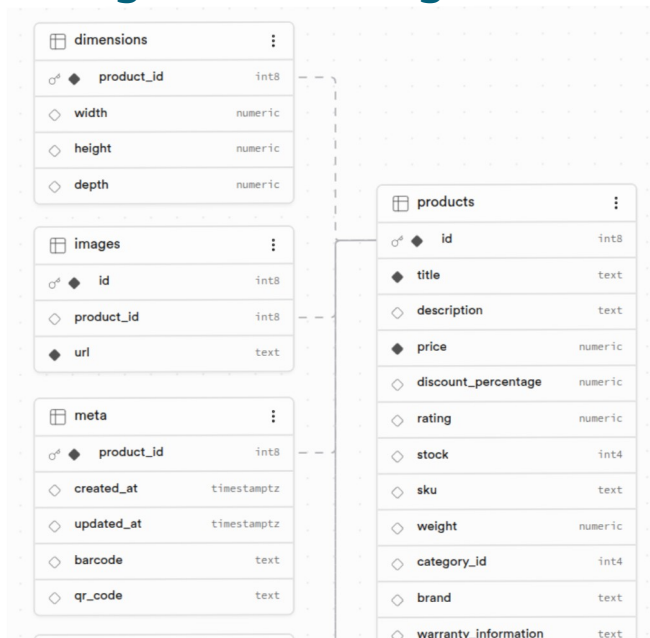
Glenns-webshop

- Komponenterna:

supabase: en databasserver som innehåller produkterna

Initialiseras genom att ange ett databas schema som sedan används för att skapa databasen

Ex



Glenns-webshop

- supabase:

komplett backend som bygger på postgresQL

Definiera ett databas schema → sätt upp databasen

Supabase ger tillgång till ett rest api som kan användas eller graphql api

Live update är möjlig: `supabase.channel("table changes")`

Authentisering kommer med, behöver inte skapa ett databas för det

Restriktioner med avseende på bandbred, antal requests etc

Nackdel med supabase: komplexa databaser, tung trafik

Glenns-webshop

- Komponenterna:

supabase: denna databas kommer man åt genom att använda sig av `const supabase =`
`createClient(process.env.NEXT_PUBLIC_SUPABASE_URL!,`
`process.env.NEXT_PUBLIC_SUPABASE_KEY!);`

Detta skapar en supabase klient som kan använda

```
const { data, error } = await supabase.from("products").select("*").limit(100);
```

`createClient()` = öppna databasen
`supabase.from()` = välj tabell
`select()` = hämta rader

Glenns-webshop

- Supabase sammanfattning:

Supabase: -> postgresSQL databas: Tillgängligt API → rest /rest/products/ : Autentisering

Client → `const supabase = createClient(url, key)`

Schema → supabase → genererar API (REST) → js client (createclient) → frontend (Nextjs)

Schema + json fil → supabase: → genererar en tabell i postgresSQL

Ex: `const { data } = await supabase.from("products").select("*")`

`<ProductGrid products={products} />` products=Supabase DB → API → JSON → React → HTML

Serverside: `page.tsx` -> `if (query) {supabaseQuery = supabaseQuery.or(`title.ilike.${query}`,description.ilike.${query}`);}` q → supabase → filter

Clientside: `product-page-client.tsx` → `<Search2 products={products} onSearch={({results}) => { setFilteredProducts(results);}} />` in mem → react filter

Client → filtrera data som redan finns

Server → data från supabase